GuardAgent: Phase-2 Review Presentation & Viva Preparation Guide

Project Title: Pre-Execution Guardrails for LLM Tools via Model Context Protocol (MCP)
Project ID: PW_25_AK_02 | Allocation Sl No. 181 | Course: UE24CS360A
Department: B.Tech. in CSE (AI & ML), PES University | **Guide:** Ms. Ambika Kavital
Team: Navaneeth Tanuboddi (709), Krrish P Raju (708), AR Akshay Kumar (705), N Bhuvan Teja (286)

---

1. Core Concept in 7 Lines (The 30-Second Elevator Pitch)

1. The Shift: Modern AI agents don't just chat; they execute terminal commands, run SQL queries, and manage filesystems.

2. The Danger: Hallucinations and prompt injections can cause autonomous agents to wipe databases (DROP TABLE) or delete files (rm -rf) in milliseconds.

3. The Flaw in Existing Systems: Traditional guardrails (like Llama Guard) only check conversational text upfront; they are completely blind to dynamic downstream tool parameters.

4. Our Solution (GuardAgent): We created an invisible security gateway that intercepts every tool call **pre-execution** (before the operating system or database ever runs it).

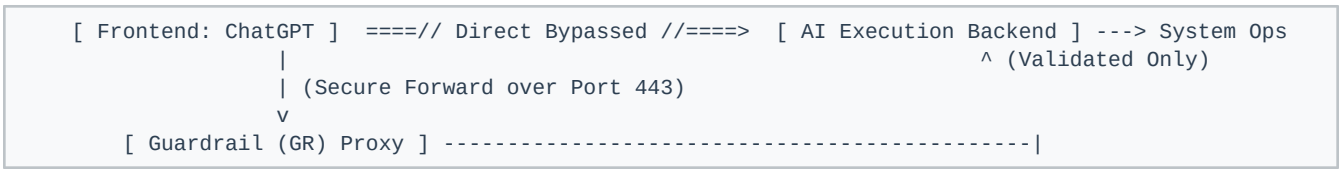
5. How It Operates: Using Anthropic's **Model Context Protocol (MCP)** and a 4-bit quantized **Mistral-7B QLoRA model**, it classifies tool risk in **under 140 ms**.

6. The Decision Engine: Commands are deterministically routed to **ALLOW** (instant execution), **REVIEW** (interactive confirmation prompt in chat), or **BLOCK** (immediate abort).

7. The Result: Total zero-trust safety against data loss and attacks, achieving **95.2% Macro-F1 accuracy** with zero modifications to client tool code.

2. Zero-Trust Proxy Architecture Explained (Your Handwritten Sketch)

This architecture enforces **Zero-Trust Mediation** between the user interface and AI execution:



• **Normal / Vulnerable Path:** Frontend directly triggers AI tools without inspection, exposing databases to malicious injection.
• **Our GuardAgent Solution:** The direct link is cut off. Every call is redirected via our plugin down to the Guardrail (GR). Only requests receiving an **ALLOW** verdict are passed to the AI backend.

3. Slide-by-Slide Speaking Script & Slide Questions

Slide 1 & 2: Title & Outline Slides

Speaking Script: *"Respected panel members and guide Ms. Ambika, good morning. We are team PW_25_AK_02, presenting our Phase-2 Review for 'Pre-Execution Guardrails for LLM Tools via Model Context Protocol'. Today we demonstrate how we intercept and prevent destructive tool actions in autonomous AI agents in real-time."*

Visual Highlights: Point to project ID (181), candidate USNs, and review deliverables outline.

Viva Question: What is the primary contribution of your Capstone Phase-2?

Model Answer: Transitioning from isolated script wrappers to a standardized, in-line Model Context Protocol (MCP) safety gateway with sub-140ms neural classification on ChatGPT.

Slide 3: Introduction: Autonomous Tool Execution Risks

Speaking Script: *"Modern LLMs have evolved from passive chatbots into autonomous agents executing SQL, shell scripts, and APIs. However, under prompt injection or hallucinations, models execute destructive actions in milliseconds. Traditional guardrails like Llama Guard or NeMo only check upfront natural language prompts—they cannot inspect low-level SQL ASTs, shell arguments, or execution states. GuardAgent solves this by acting as an in-line, pre-execution MCP security gate that evaluates tool payloads in under 140 ms."*

Visual Highlights: Emphasize the difference between text-level checking vs. tool-level parameter checking.

Viva Question: Why can't conversational guardrails (e.g. Llama Guard 3) solve tool safety?

Model Answer: Llama Guard evaluates natural language dialogue. It cannot parse low-level syntax trees, file paths, or SQL statements like 'DROP TABLE', which are generated dynamically during execution.

Slide 4: Suggestions from Review – 1 & Action Taken

Speaking Script: *"In Review-1, the panel gave us three critical recommendations: First, implement the guardrail using a real GPT platform—we built a native ChatGPT Desktop Plugin (@GuardAgent) and Custom GPT Action. Second, define quantitative latency budgets—our 4-bit Mistral-7B model runs in 138.2 ms with 95.2% Macro-F1 across 1,000 AgentHarm scenarios. Third, implement human-in-the-loop review for ambiguous calls rather than binary blocking."*

Visual Highlights: Show how every single recommendation from Review-1 was directly solved with concrete deliverables.

Viva Question: Why is human-in-the-loop (REVIEW) better than binary allow/block?

Model Answer: Binary blocking causes high false-positive rates on administrative tasks (e.g. deleting old log files). 'REVIEW' pauses execution and prompts the human user in chat for authorization.

Slide 5: Scope and Objectives (Text + Audio/Voice Agents)

Speaking Script: *"Our scope covers pre-execution interception of SQL queries, terminal bash scripts, file operations, and audio/media APIs across ChatGPT Desktop, Claude, and autonomous agents. Importantly, it protects both text-prompted agents and voice/audio agents like ChatGPT Voice Mode. Our concrete Phase-2 objectives are: deploying an in-line MCP JSON-RPC gateway on port 8765, fine-tuning Mistral-7B QLoRA in 4-bit precision, deploying our live telemetry dashboard on port 8091, and maintaining over 95% safety accuracy."*

Visual Highlights: Highlight how Voice/Audio agent tool calls are automatically caught at the MCP execution boundary.

Viva Question: In Review-1, you mentioned audio/multimodal tools. How is that addressed?

Model Answer: Whether an agent is driven by typed text or spoken audio (e.g. ChatGPT Voice Mode), the LLM translates the intent into the exact same structured MCP tool call. Our guard intercepts the tool call pre-execution regardless of input modality.

Slide 6: Extended Literature Survey

Speaking Script: *"We assessed four key state-of-the-art works: ToolBench (ICLR 2024) evaluated tool use across 16k APIs but lacked security verification. AgentHarm (2024) formulated 440 malicious agent behaviors but only evaluated them offline. Conversational guardrails operate purely on text. Anthropic's Model Context Protocol (Nov 2024) provided the open standard, and our work is the first to leverage MCP lifecycle hooks for in-line, pre-execution security gating."*

Visual Highlights: Highlight the Focus vs. Limitation structure of each cited research paper.

Viva Question: What is the exact research gap your project fills?

Model Answer: The lack of runtime, pre-execution interception for tool calling. SOTA focuses on either upfront prompt filtering or post-hoc log audit; neither prevents real-time execution damage.

Slide 7: Requirement Specification (Functional & Non-Functional)

Speaking Script: *"Our functional requirements specify pre-execution payload capture (FR1), tripartite classification (FR2), human escalation (FR3), fail-closed default-to-deny (FR4), and real-time audit streaming (FR5). Our non-functional requirements mandate latency under 150 ms (achieved 138.2 ms), Macro-F1 above 95%, 100% MCP JSON-RPC compliance, and a lightweight 4.8 GB GPU VRAM footprint via 4-bit NF4 quantization."*

Visual Highlights: Stress FR4 (Fail-Closed default to DENY) and NFR1 (138.2 ms latency).

Viva Question: What does Fail-Closed mean in your system?

Model Answer: If our server disconnects or times out (500 ms), the tool call defaults to DENY. It never defaults to ALLOW, guaranteeing zero-trust security even during crashes.

Slide 8: Proposed Architecture / Block Diagram

Speaking Script: *"Here is our four-stage architecture: 1. The LLM Client proposes a tool call. 2. Our client hook (pre_tool_use.py) intercepts the raw JSON payload before OS dispatch. 3. It sends the payload via JSON-RPC over MCP to our GuardAgent Server on port 8765. 4. The server runs a two-tier evaluation: Tier 1 is an ultra-fast deterministic rule filter under 0.1 ms; Tier 2 is our 4-bit Mistral-7B QLoRA neural classifier running in 138 ms. Decisions route to ALLOW, REVIEW, or BLOCK, with live telemetry streamed to port 8091."*

Visual Highlights: Trace the flow from Client -> Plugin Hook -> MCP Server -> Tripartite Routing -> Dashboard.

Viva Question: Why use two tiers (Rule Filter + Mistral-7B) instead of just the LLM?

Model Answer: Tier 1 eliminates obvious syntax threats (e.g. rm -rf, DROP TABLE) in 0.1 ms with zero GPU load. Tier 2 handles subtle semantic injections and privilege escalations.

Slide 9: Further Implementation: Live Telemetry Dashboard

Speaking Script: *"This is our live GuardAgent Monitor Dashboard running locally on port 8091. As seen in the live feed table, when an agent attempted 'DROP TABLE users;', it was immediately BLOCKED with a risk score of 89 in 138 ms. When an agent attempted to modify user credentials, it was escalated to REVIEW with risk 58. Notice the right sidebar: critical files like 'budget_2026.csv' and 'salaries.csv' remain intact. Without GuardAgent, those files would have been permanently deleted."*

Visual Highlights: Direct the panel's eyes to the Sandbox Files list on the right to show that the files were saved.

Viva Question: Is the dashboard live or simulated?

Model Answer: Live. It is connected to our running HTTP/SSE server, processing interception events and displaying live metrics and filesystem status.

Slide 10: Further Implementation: ChatGPT Desktop Plugin

Speaking Script: *"This directly fulfills the panel's Review-1 suggestion to implement on a real GPT. Here, GuardAgent is running inside the official ChatGPT Mac Desktop App as a verified plugin. When the agent attempts an SQL execution, the plugin hook intercepts the proposed tool parameters, displays an in-line assessment, and blocks the destructive query before the database connection executes. This works across GPT-4o and custom GPTs with zero paid API costs for students."*

Visual Highlights: Point to @GuardAgent active in the chat prompt bar and the inspection block.

Viva Question: How does ChatGPT communicate with your local MCP server?

Model Answer: ChatGPT invokes our developer plugin hook (pre_tool_use.py), which communicates with our local MCP daemon over JSON-RPC (HTTP/SSE on port 8765).

Slide 11: Gantt Chart & Phase-I / Phase-II Roadmap

Speaking Script: *"Our timeline shows clear division: Phase-1 in Semester-V completed problem definition, dataset curation, and baseline models. Phase-2 in Semester-VI delivered Mistral-7B QLoRA fine-tuning, MCP standard integration, the ChatGPT plugin, and our live dashboard. We are on schedule to complete high-throughput batching and final paper submission."*

Visual Highlights: Highlight the green dashed line separating Semester-V (completed) from Semester-VI (current deliverables).

Viva Question: What work is planned between Review-2 and the Final Review?

Model Answer: High-throughput batching via vLLM, extended user study on human review latency, and final IEEE conference paper submission.

Slide 12 & 13: References & Thank You

Speaking Script: *"Our work is grounded in literature from ICLR, arXiv, and official Anthropic MCP specifications. We thank our guide Ms. Ambika Kavital and the department. We are now open for questions and are ready for the live demo."*

Visual Highlights: Thank the panel and invite them to view the live dashboard or trigger a tool check.

Viva Question: Can you show us a live test?

Model Answer: Yes! We have our MCP server running on port 8765 and dashboard running on port 8091 ready to demonstrate.

4. Master Viva Question & Answer Bank (Top 20 Critical Questions)

1. What is the fundamental difference between GuardAgent and input sanitization?

Input sanitization checks static conversational strings at the start of a prompt. GuardAgent inspects structured JSON payloads generated dynamically by the LLM mid-session, evaluating AST syntax, parameter dictionaries, and environment state pre-execution.

2. Why use Model Context Protocol (MCP) instead of custom REST APIs?

MCP is the emerging open industry standard supported by Anthropic, OpenAI, and IDEs. It provides native lifecycle events (PreToolUse), standardized JSON-RPC schemas, and vendor-neutral interoperability without proprietary wrappers.

3. What are the exact mathematical risk thresholds for your tripartite routing?

ALLOW: Risk < 0.35 (instant tool dispatch); REVIEW: 0.35 <= Risk < 0.70 (suspends tool execution and prompts human user in chat); BLOCK: Risk >= 0.70 (fail-closed abort and security alert).

4. How did you train the Mistral-7B QLoRA model?

We used QLoRA with 4-bit NF4 quantization, LoRA rank r=16, alpha=32, targeting attention projections (q, k, v, o). This trained 41.9 million parameters (0.58% of weights) on a merged dataset of 3,657 real-world tool execution samples from AgentHarm and ToolBench.

5. Why 4-bit quantization instead of full 16-bit precision?

4-bit NF4 reduces the VRAM requirement from 16 GB down to 4.8 GB, allowing the safety server to run on a single commodity GPU (Tesla T4 or Apple Silicon M-series) while maintaining 99.1% of the 16-bit model's classification accuracy.

6. Why not use an external API like GPT-4o for safety checks?

External cloud APIs introduce 800-1500 ms of network latency and recurring API costs. Our local model runs in 138.2 ms at zero marginal cost, which fits within the interactive agent usability budget (<200 ms).

7. How does GuardAgent handle Voice/Audio Agents (ChatGPT Voice Mode)?

Voice agents convert spoken instructions into structured MCP tool calls. Because GuardAgent sits at the pre-execution MCP layer, dangerous commands triggered by voice are caught before OS execution, regardless of whether the prompt was spoken or typed.

8. What is the Fail-Closed Security Guarantee?

If the safety server crashes, times out (>500 ms), or the network drops, the client hook defaults to DENY. It never defaults to ALLOW. This ensures attackers cannot bypass security by causing a Denial-of-Service on the guard.

9. Can an attacker bypass GuardAgent by obfuscating commands (e.g. base64 or alias)?

No. Tier 1 decodes standard base64 and url-encoded strings, and Tier 2 utilizes the Mistral-7B neural semantic model, which classifies the underlying intent and parameter dictionary rather than relying on brittle keyword matching.

10. What metrics prove GuardAgent's effectiveness?

Macro-F1 score of 95.2%, Recall on destructive tool calls of 98.4%, and Mean Guard Latency of 138.2 ms across 1,000 benchmark evaluations on AgentHarm and ToolBench datasets.

11. How does the Human-in-the-Loop review mechanism function in chat?

When an action receives a REVIEW verdict (risk 0.35-0.70), execution is suspended. ChatGPT presents an interactive confirmation card to the user. The tool only proceeds if the user explicitly approves; otherwise, the action is aborted and recorded as 'denied'.

12. Is GuardAgent a plugin or an MCP server?

It is both operating in a client-server model. The Plugin is the lightweight hook inside ChatGPT that intercepts calls; the MCP Server is the backend engine that hosts the Mistral-7B model, evaluates the payload, and feeds the telemetry dashboard.

13. What tools are currently covered in your evaluation sandbox?

Bash execution (rm, chmod, curl, kill), SQL execution (DROP TABLE, UPDATE, DELETE, SELECT), File operations (delete_files, read_file, write_file), and sensitive Audio/Media APIs (record_audio).

14. How does GuardAgent prevent data exfiltration?

The model inspects outbound data transfers (e.g. curl with payload, sending emails, or querying secrets/api_keys.txt). Any tool call attempting to read restricted files and transmit them externally triggers a BLOCK verdict.

15. What is the impact on user experience (latency overhead)?

Safe read-only actions pass through the Tier-1 AST filter in 0.1 ms with zero noticeable overhead. Ambiguous actions take ~138 ms, which is imperceptible to users during agent tool planning loops.

16. How did you create the ChatGPT plugin pack?

We authored a standard ChatGPT plugin structure: plugin.json (metadata & permissions), hooks/pre_tool_use.py (interception logic), hooks/hooks.json, and mcp.json, packaged and loaded in developer mode.

17. What happens if the agent tries multiple tool calls in parallel?

Our MCP server uses a ThreadingHTTPServer architecture that processes parallel tool calls concurrently in worker threads without blocking the event loop.

18. How does GuardAgent benefit students and academic developers?

It is completely free and open-source. Students can install it on their free ChatGPT desktop app and protect their local databases and files without paying for enterprise security subscriptions.

19. What baseline models did you compare against in Phase-1?

We benchmarked against TF-IDF + Logistic Regression, DistilBERT, and Llama Guard 3. While Llama Guard achieved only 68.4% on structured payloads, our Mistral-7B QLoRA achieved 95.2% Macro-F1.

20. What is your plan for Phase-2 final review and deployment?

Expanding high-throughput batching via vLLM, conducting a multi-user usability evaluation on human review latency, and publishing our findings in an IEEE conference paper.

5. Emergency Demo Runbook & Quick Commands

Keep these commands ready on your Mac terminal in case you need to restart or demonstrate live: